Optimization and Tuning

This guide covers optimization strategies and performance tuning for vLLM V1.

> **Tip**
>
> Running out of memory? Consult this guide on how to conserve memory.

Optimization Levels

vLLM provides 4 optimization levels (`-O0` , `-O1` , `-O2` , `-O3`) that allow users to trade off startup time for performance:

- `-O0` : No optimizations. Fastest startup time, but lowest performance.
- `-O1` : Fast optimization. Simple compilation and fast fusions, and PIECEWISE cudagraphs.
- `-O2` : Default optimization. Additional compilation ranges, additional fusions, FULL_AND_PIECEWISE cudagraphs.
- `-O3` : Aggressive optimization. Currently equal to `-O2` , but may include additional time-consuming or experimental optimizations in the future.

For more information, see the optimization level documentation.

Preemption

Due to the autoregressive nature of transformer architecture, there are times when KV cache space is insufficient to handle all batched requests. In such cases, vLLM can preem KV cache space for other requests. Preempted requests are recomputed wh e space becomes available again. When this occurs, you may see the following warning:

Ask AI

stable

```
WARNING 05-09 00:49:33 scheduler.py:1057 Sequence group 0 is pr
PreemptionMode.RECOMPUTE mode because there is not enough KV cache space. This can
affect the end-to-end performance. Increase gpu_memory_utilization or
```

```
tensor_parallel_size to provide more KV cache memory.
total_cumulative_preemption_cnt=1
```

While this mechanism ensures system robustness, preemption and recomputation can adversely affect end-to-end latency. If you frequently encounter preemptions, consider the following actions:

- Increase `gpu_memory_utilization`. vLLM pre-allocates GPU cache using this percentage of memory. By increasing utilization, you can provide more KV cache space.
- Decrease `max_num_seqs` or `max_num_batched_tokens`. This reduces the number of concurrent requests in a batch, thereby requiring less KV cache space.
- Increase `tensor_parallel_size`. This shards model weights across GPUs, allowing each GPU to have more memory available for KV cache. However, increasing this value may cause excessive synchronization overhead.
- Increase `pipeline_parallel_size`. This distributes model layers across GPUs, reducing the memory needed for model weights on each GPU, indirectly leaving more memory available for KV cache. However, increasing this value may cause latency penalties.

You can monitor the number of preemption requests through Prometheus metrics exposed by vLLM. Additionally, you can log the cumulative number of preemption requests by setting `disable_log_stats=False`.

In vLLM V1, the default preemption mode is `RECOMPUTE` rather than `SWAP`, as recomputation has lower overhead in the V1 architecture.

Chunked Prefill

Chunked prefill allows vLLM to process large prefills in smaller chunks and batch them together with decode requests. This feature helps improve both throughput and latency by better balancing compute-bound (prefill) and memory-bound (decode) operations.

In V1, **chunked prefill is enabled by default whenever possible**. With chunked prefill enabled, the scheduling policy prioritizes decode requests. It batches all pending decode requests before scheduling any prefill operations. When there are available tokens in the `max_` ns budget, it schedules pending prefills. If a pending prefill request cannot fit in `max_num_batched_tokens`, it automatically chunks it.

Ask AI

This policy has two benefits:

stable

- It improves ITL and generation decode because decode requests are p

- It helps achieve better GPU utilization by locating compute-bound (prefill) and memory-bound (decode) requests to the same batch.

Performance Tuning with Chunked Prefill

You can tune the performance by adjusting `max_num_batched_tokens`:

- Smaller values (e.g., 2048) achieve better inter-token latency (ITL) because there are fewer prefills slowing down decodes.
- Higher values achieve better time to first token (TTFT) as you can process more prefill tokens in a batch.
- For optimal throughput, we recommend setting `max_num_batched_tokens > 8192` especially for smaller models on large GPUs.
- If `max_num_batched_tokens` is the same as `max_model_len`, that's almost the equivalent to the V0 default scheduling policy (except that it still prioritizes decodes).

> **Warning**
>
> When chunked prefill is disabled, `max_num_batched_tokens` must be greater than `max_model_len`. In that case, if `max_num_batched_tokens < max_model_len`, vLLM may crash at server start-up.

```
from vllm import LLM

# Set max_num_batched_tokens to tune performance
llm = LLM(model="meta-llama/Llama-3.1-8B-Instruct", max_num_batched_tokens=16384)
```

See related papers for more details (https://arxiv.org/pdf/2401.08671 or https://arxiv.org/pdf/2308.16369).

Parallelism Strategies

vLLM supports multiple parallelism strategies that can be combined to optimize across different hardware configurations.

Ask AI

Tensor Parallelism (TP)

stable

Tensor parallelism shards model parameters across multiple GPUs within each model layer. This is the most common strategy for large model inference within a single node.

When to use:

- When the model is too large to fit on a single GPU
- When you need to reduce memory pressure per GPU to allow more KV cache space for higher throughput

```
from vllm import LLM

# Split model across 4 GPUs
llm = LLM(model="meta-llama/Llama-3.3-70B-Instruct", tensor_parallel_size=4)
```

For models that are too large to fit on a single GPU (like 70B parameter models), tensor parallelism is essential.

Pipeline Parallelism (PP)

Pipeline parallelism distributes model layers across multiple GPUs. Each GPU processes different parts of the model in sequence.

When to use:

- When you've already maxed out efficient tensor parallelism but need to distribute the model further, or across nodes
- For very deep and narrow models where layer distribution is more efficient than tensor sharding

Pipeline parallelism can be combined with tensor parallelism for very large models:

```
from vllm import LLM

# Combine pipeline and tensor parallelism
llm = LLM(
    model="meta-llama/Llama-3.3-70B-Instruct,
    tensor_parallel_size=4,
    pipeline_parallel_size=2,
)
```

Ask AI

Expert Parallelism (EP)

Expert parallelism is a specialized form of parallelism for Mixture of Expert
different expert networks are distributed across GPUs.

stable

When to use:

- Specifically for MoE models (like DeepSeekV3, Qwen3MoE, Llama-4)
- When you want to balance the expert computation load across GPUs

Expert parallelism is enabled by setting `enable_expert_parallel=True`, which will use expert parallelism instead of tensor parallelism for MoE layers. It will use the same degree of parallelism as what you have set for tensor parallelism.

Data Parallelism (DP)

Data parallelism replicates the entire model across multiple GPU sets and processes different batches of requests in parallel.

When to use:

- When you have enough GPUs to replicate the entire model
- When you need to scale throughput rather than model size
- In multi-user environments where isolation between request batches is beneficial

Data parallelism can be combined with the other parallelism strategies and is set by `data_parallel_size=N`. Note that MoE layers will be sharded according to the product of the tensor parallel size and data parallel size.

Batch-level DP for Multi-Modal Encoders

By default, TP is used to shard the weights of multi-modal encoders just like for language decoders, in order to reduce the memory and compute load on each GPU.

However, since the size of multi-modal encoders is very small compared to language decoders, there is relatively little gain from TP. On the other hand, TP incurs significant communication overhead because of all-reduce being performed after every layer.

Given this, it may be advantageous to instead shard the batched input data using TP, essentially performing batch-level DP. This has been shown to improve the throughput and TTFT by around 10% for `tensor_parallel_size=8`. For vision encoders that use hardware-u
operations, batch-level DP can provide another 40% improvement compared

Ask AI

Nevertheless, since the weights of the multi-modal encoder are replicated across each TP rank, there will be a minor increase in memory consumption and may cause OOM if you can barely fit the model already.

stable

You can enable batch-level DP by setting `mm_encoder_tp_mode="data"`, for example:

```python
from vllm import LLM

llm = LLM(
    model="Qwen/Qwen2.5-VL-72B-Instruct",
    tensor_parallel_size=4,
    # When mm_encoder_tp_mode="data",
    # the vision encoder uses TP=4 (not DP=1) to shard the input data,
    # so the TP size becomes the effective DP size.
    # Note that this is independent of the DP size for language decoder which is
used in expert parallel setting.
    mm_encoder_tp_mode="data",
    # The language decoder uses TP=4 to shard the weights regardless
    # of the setting of mm_encoder_tp_mode
)
```

Important

Batch-level DP is not to be confused with API request-level DP (which is instead controlled by `data_parallel_size`).

Batch-level DP needs to be implemented on a per-model basis, and enabled by setting `supports_encoder_tp_data = True` in the model class. Regardless, you need to set `mm_encoder_tp_mode="data"` in engine arguments to use this feature.

Known supported models (with corresponding benchmarks):

- dots_ocr (Pull Request #25466)
- GLM-4.1V or above (Pull Request #23168)
- InternVL (Pull Request #23909)
- Kimi-VL (Pull Request #23817)
- Llama4 (Pull Request #18368)
- MiniCPM-V-2.5 or above (Pull Request #23327, Pull Request #23948)
- Qwen2-VL or above (Pull Request #22742, Pull Request #24955, Pull Request #25445)
- Step3 (Pull Request #22697)

Ask AI

Input Processing

stable

Parallel Processing

You can run input processing in parallel via API server scale-out. This is useful when input processing (which is run inside the API server) becomes a bottleneck compared to model execution (which is run inside engine core) and you have excess CPU capacity.

```
# Run 4 API processes and 1 engine core process
vllm serve Qwen/Qwen2.5-VL-3B-Instruct --api-server-count 4

# Run 4 API processes and 2 engine core processes
vllm serve Qwen/Qwen2.5-VL-3B-Instruct --api-server-count 4 -dp 2
```

Note

API server scale-out is only available for online inference.

Warning

By default, 8 CPU threads are used in each API server to load media items (e.g. images) from request data.

If you apply API server scale-out, consider adjusting `VLLM_MEDIA_LOADING_THREAD_COUNT` to avoid CPU resource exhaustion.

Note

API server scale-out disables multi-modal IPC caching because it requires a one-to-one correspondence between API and engine core processes.

This does not impact multi-modal processor caching.

Multi-Modal Caching

Multi-modal caching avoids repeated transfer or processing of the same mu
commonly occurs in multi-turn conversations.

Ask AI

Processor Caching

stable

Multi-modal processor caching is automatically enabled to avoid repeated., processing the same multi-modal inputs in `BaseMultiModalProcessor`.

IPC Caching

Multi-modal IPC caching is automatically enabled when there is a one-to-one correspondence between API (`P0`) and engine core (`P1`) processes, to avoid repeatedly transferring the same multi-modal inputs between them.

Key-Replicated Cache

By default, IPC caching uses a **key-replicated cache**, where cache keys exist in both the API (`P0`) and engine core (`P1`) processes, but the actual cache data resides only in `P1` .

Shared Memory Cache

When multiple worker processes are involved (e.g., when TP > 1), a **shared-memory cache** is more efficient. This can be enabled by setting `mm_processor_cache_type="shm"` . In this mode, cache keys are stored on `P0` , while the cache data itself lives in shared memory accessible by all processes.

Configuration

You can adjust the size of the cache by setting the value of `mm_processor_cache_gb` (default 4 GiB).

If you do not benefit much from the cache, you can disable both IPC and processor caching completely via `mm_processor_cache_gb=0` .

Examples:

Ask AI

stable

```
# Use a larger cache
llm = LLM(
    model="Qwen/Qwen2.5-VL-3B-Instruct",
    mm_processor_cache_gb=8,
)

# Use a shared-memory based IPC cache
llm = LLM(
    model="Qwen/Qwen2.5-VL-3B-Instruct",
    tensor_parallel_size=2,
    mm_processor_cache_type="shm",
    mm_processor_cache_gb=8,
)

# Disable the cache
llm = LLM(
    model="Qwen/Qwen2.5-VL-3B-Instruct",
    mm_processor_cache_gb=0,
)
```

Cache Placement

Based on the configuration, the content of the multi-modal caches on `P0` and `P1` are as follows:

mm_processor_cache_type	Cache Type	`P0` Cache	`P1` Engine Cache	`P1` Worker Cache	M M
lru	Processor Caching	K + V	N/A	N/A	
lru	Key-Replicated Caching	K	K + V	N/A	

Ask AI

stable

mm_processor_cache_type	Cache Type	`P0` Cache	`P1` Engine Cache	`P1` Worker Cache	M M
shm	Shared Memory Caching	K	N/A	V	m s - a r
N/A	Disabled	N/A	N/A	N/A	6

K: Stores the hashes of multi-modal items V: Stores the processed tensor data of multi-modal items

CPU Resources for GPU Deployments

vLLM V1 uses a multi-process architecture (see V1 Process Architecture) where each process requires CPU resources. Underprovisioning CPU cores is a common source of performance degradation, especially in virtualized environments.

Minimum CPU Requirements

For a deployment with `N` GPUs, there are at minimum:

- **1 API server process** -- handles HTTP requests, tokenization, and input processing
- **1 engine core process** -- runs the scheduler and coordinates GPU workers
- **N GPU worker processes** -- one per GPU, executes model forward passes

This means there are always at least `2 + N` **processes** competing for CPU time.

Warning

Ask AI

Using fewer physical CPU cores than processes will cause contention and significantly degrade throughput and latency. The engine core process runs a busy loop and is particularly sensitive to CPU starvation.

stable

The minimum is `2 + N` physical cores (1 for the API server, 1 for the engine core, and 1 per GPU worker). In practice, allocating more cores improves performance because the OS, PyTorch background threads, and other system processes also need CPU time.

> **Important**
>
> Please note we are referring to **physical CPU cores** here. If your system has hyperthreading enabled, then 1 vCPU = 1 hyperthread = 1/2 physical CPU core, so you need `2 x (2 + N)` minimum vCPUs.

Data Parallel and Multi-API Server Deployments

When using data parallelism or multiple API servers, the CPU requirements increase:

```
Minimum physical cores = A + DP + N + (1 if DP > 1 else 0)
```

where `A` is the API server count (defaults to `DP`), `DP` is the data parallel size, and `N` is the total number of GPUs. For example, with `DP=4, TP=2` on 8 GPUs:

```
4 API servers + 4 engine cores + 8 GPU workers + 1 DP coordinator = 17 processes
```

Performance Impact

CPU underprovisioning particularly impacts:

- **Input processing throughput** -- tokenization, chat template rendering, and multi-modal data loading all run on CPU
- **Scheduling latency** -- the engine core scheduler runs on CPU and directly affects how quickly new tokens are dispatched to the GPU workers
- **Output processing** -- detokenization, networking, and especially streaming token responses use CPU cycles

If you observe that GPU utilization is lower than expected, CPU contention m
Increasing the number of available CPU cores and even the clock speed car
end-to-end performance.

Ask AI

Attention Backend Selection

stable

vLLM supports multiple attention backends optimized for different hardware and use cases. The backend is automatically selected based on your GPU architecture, model type, and configuration, but you can also manually specify one for optimal performance.

For detailed information on available backends, their feature support, and how to configure them, see the Attention Backend Feature Support documentation.

March 9, 2026

Ask AI

stable